

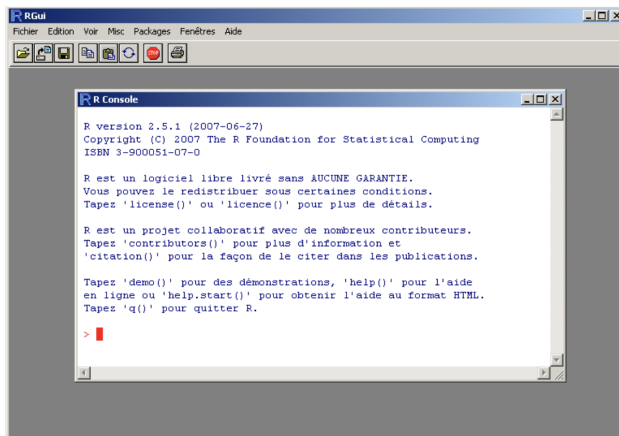
# Initiation à R : prise en main

## Introduction aux logiciels R et RStudio

Le logiciel R est un logiciel gratuit qui permet d'effectuer de nombreux calculs et tests statistiques. Il est téléchargeable sur la page <http://cran.r-project.org>.

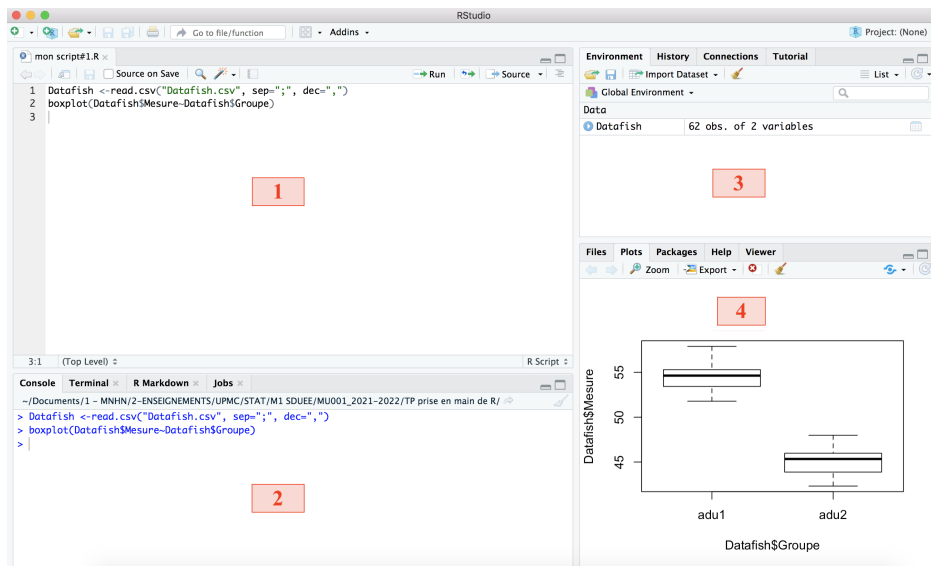
Veillez à choisir l'installateur correspondant à votre système d'exploitation.

Voici à quoi va ressembler l'interface de R sous Windows :



Comme R est logiciel qui peut paraître aride, a été ajouté autour de lui un environnement pour le rendre un peu plus convivial. C'est le logiciel RStudio, que nous emploierons pour ce TP. Lui aussi est gratuit. Comme RStudio est un environnement de développement intégré pour R (IDE), il faut avoir installer R avant d'installer RStudio. On trouve RStudio sur : <http://www.rstudio.com/products/rstudio/download>

L'interface de RStudio se découpe en 4 fenêtres.

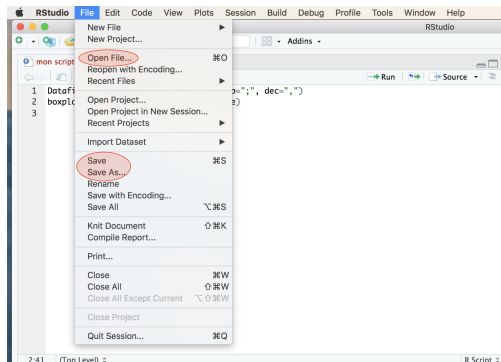


## Fenêtre 1.

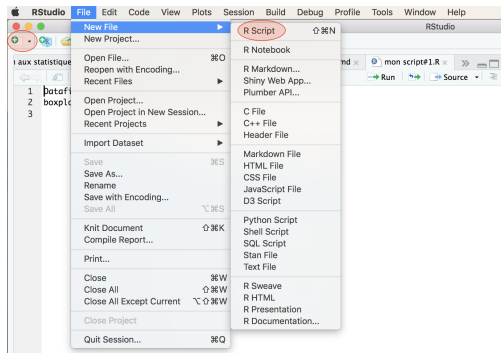
Le fichier que vous êtes en train de lire est un **script**. C'est un éditeur de texte, qui va contenir l'ensemble des commandes que R peut exécuter.

R ignorera tout ce qui est situé derrière un **#** dans une ligne donnée. Cela s'appelle un **commentaire**. Les instructions pour le TP seront donc situées dans ce script, sous forme de commentaires. Vous pourrez aussi annoter les scripts du TP avec vos propres commentaires en les faisant précéder d'un **#**.

Ils peuvent s'enregistrer (ils le sont avec l'extension .R), en cliquant en haut à gauche sur *>File >Save (ou Save as)* ou directement sur l'icône disquette. Ils peuvent se ré-ouvrir en cliquant sur *>File >Open File*.



Pour ouvrir un nouveau script, cliquer sur *>File >New File > R Script* (ou directement sur l'icône + en haut à gauche).



## Fenêtre 2.

En bas à gauche de R studio se trouve la **console**, qui est une fenêtre de commandes où s'afficheront les commandes exécutées et leurs résultats (si un résultat doit s'afficher).

Le plus simple est d'exécuter, à partir du script, les commandes, qui s'afficheront ensuite automatiquement dans la console.

**Pour exécuter une commande à partir du script :** sélectionner la ou les lignes à exécuter dans la fenêtre script, puis appuyer sur les touches ctrl + R (version PC) ou cmd + enter (version Mac).

Vous pouvez aussi sélectionner les lignes de commandes dans le script et cliquer sur "Run" en haut à droite de la fenêtre 1.



### Fenêtre 3.

En haut à droite, vous trouverez l'onglet *Environment*, qui liste l'ensemble des éléments que vous avez créés ou importés, et sur lesquels vous pouvez travailler. A chaque ouverture de jeu de données, le nom et la description de ces données seront affichés ici. L'onglet *History* récapitule tout ce qui a été fait et exécuté.

### Fenêtre 4.

En bas à droite, l'onglet *Files* est un navigateur de fichiers pour sélectionner les scripts R à éditer ou les fichiers de données à importer. L'onglet *Plots* regroupe les graphiques réalisés; vous pourrez sauvegarder vos graphes à partir de cet onglet. L'onglet *Packages* regroupe les différentes bibliothèques de R installées (à l'installation de R, la bibliothèque de base est automatiquement installée, et d'autres pourront être installées en fonction des besoins, lors de vos analyses statistiques ou graphiques). L'onglet *Help* sert à chercher de l'aide sur des commandes et trouver la documentation de R.

---

## 1<sup>ère</sup> partie : Commandes simples

### Premières commandes : R est une calculatrice

Exécutez les commandes suivantes à partir de la fenêtre de script (en les sélectionnant puis ctrl + R ou cmd + enter, ou en cliquant sur "Run").

```
1+3
```

```
## [1] 4
```

```
4/789
```

```
## [1] 0.005069708
```

```
pi
```

```
## [1] 3.141593
```

```
10^3
```

```
## [1] 1000
```

Voyons un cas particulier :

```
1/0
```

```
## [1] Inf
```

On nous signale que c'est l'infini (*Inf*).

### Les fonctions

De façon plus intéressante, R va reconnaître des **fonctions** :

```
sqrt(81) # square-root (racine carree)
```

```
## [1] 9
```

```
exp(1) # exponentiel
```

```
## [1] 2.718282
```

```
log(10) # logarithme naturel
```

```
## [1] 2.302585
```

```
log10(10) # logarithme decimal
```

```
## [1] 1
```

```
#
```

```
log(exp(3)) # Il est tout à fait possible de combiner les fonctions
```

```
## [1] 3
```

Les **fonctions** sont toujours suivies de **parenthèses**.

Les éléments contenus dans ces parenthèses (= ce sur quoi la fonction travaille) s'appellent des **arguments**. S'il y a plusieurs arguments, ceux-ci sont séparés par des **virgules**.

R Studio propose automatiquement une parenthèse fermante quand vous en ouvrez une, et R affiche un + dans la console quand il en manque.

Pour comprendre ce que fait une fonction et quels sont ses arguments, on peut utiliser l'aide en mettant un ? devant la fonction, ou regarder dans l'onglet Help en bas à droite (fenêtre 3), ou faire appel à la fonction `help()`.

```
?log
```

```
help(log)
```

---

## 2<sup>ème</sup> partie : Les données

### Introduction

Concrètement, comment utiliser R ?

on emploie des fonctions sur des données que l'on va maintenant apprendre à saisir.

### Vecteurs numériques

```
a <- 42 # on peut aussi faire 42 -> a ou a= 42
a
```

```
## [1] 42
```

Le signe utilisé <- est le signe d'**assignation**, et peut être remplacé par =. Il permet d'assigner (d'attribuer) la valeur 42 à l'**objet** *a*, que nous appellerons *vecteur* (numérique ici puisque ce vecteur contient une valeur numérique).

Pour créer un vecteur numérique de plusieurs valeurs, on utilise la fonction `c()` (pour "concatenate", qui signifie "enchaîner" en anglais, et permet alors d'obtenir une "chaîne" de valeurs).

```
a=c(1,2,412,4) # on peut aussi faire a<-c(1,2,412,4) c(1,2,412,4)->a
a # l'ancien a est remplacé
```

```
## [1] 1 2 412 4
```

**ATTENTION !** R différencie les majuscules et les minuscules dans les noms d'objet (source d'erreur fréquente)

```
A<-18;A #remarquez le point-virgule qui permet de séparer deux commandes différentes sur une même ligne
```

```
## [1] 18
```

Essayez maintenant d'autres commandes sur vos données contenues dans le vecteur numérique *a* :

```
a*2
```

```
## [1] 2 4 824 8
```

```
a+1
```

```
## [1] 2 3 413 5
```

```
a[3] # les crochets permettent d'accéder à un élément donné de a
```

```
## [1] 412
```

```
a[42] #le NA signifie qu'il n'y a rien
```

```
## [1] NA
```

```
sqrt(a)
```

```
## [1] 1.000000 1.414214 20.297783 2.000000
```

```
a*a
```

```
## [1] 1 4 169744 16
```

Note: lorsqu'on fait une opération sur `a`, R affiche le résultat mais ne modifie pas `a`.

```
a-5
```

```
## [1] -4 -3 407 -1
```

```
a
```

```
## [1] 1 2 412 4
```

Pour modifier 'a', il faut utiliser le signe d'assignation.

```
a = a - 5
```

```
a
```

```
## [1] -4 -3 407 -1
```

## Appliquer des fonctions sur les données

On voit que R vous permet de faire des calculs sur des données.

On va pouvoir aussi y appliquer des fonctions :

```
mean(a)
```

```
## [1] 99.75
```

```
median(a)
```

```
## [1] -2
```

```
quantile(a)
```

```
## 0% 25% 50% 75% 100%
```

```
## -4.00 -3.25 -2.00 101.00 407.00
```

```
var(a)
```

```
## [1] 41958.25
```

```
sqrt(var(a))
```

```
## [1] 204.8371
```

```
sd(a) #sd pour standard deviation = écart-type en anglais
```

```
## [1] 204.8371
```

```
max(a)
```

```
## [1] 407
```

```
min(a)
```

```
## [1] -4
```

```
summary(a)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      -4.00  -3.25   -2.00   99.75  101.00   407.00
```

## Vecteurs de caractères

Enfin, rien ne vous empêche de manipuler des lettres, cela sera utile pour les variables catégorielles (qualitatives).

```
z=c("a","a")
```

```
z
```

```
## [1] "a" "a"
```

L'objet *z* est un vecteur de caractères (*chr*).

```
# Remarque : comparer z et x :
```

```
x=c(a,a)
```

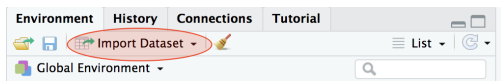
```
x
```

```
## [1] -4 -3 407 -1 -4 -3 407 -1
```

## Importer des données à partir d'un fichier

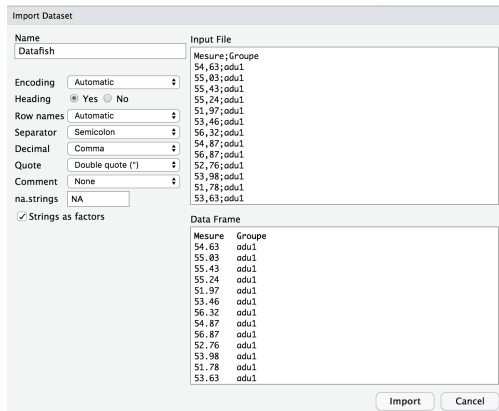
On peut aussi lire des données à partir d'un fichier de format *.csv* ou *.txt* (facile à créer avec Excel, par exemple). C'est certainement ce que vous ferez le plus. Il existe de nombreuses façons d'importer des données dans R Studio, on vous en présente ici quelques unes.

### A partir de l'onglet Environment, puis Import Dataset



#### \* En choisissant From Text (base)

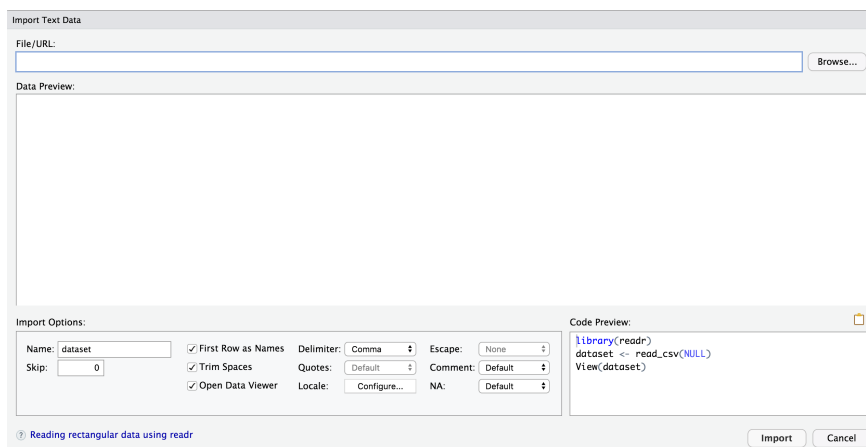
Un navigateur de fichiers s'ouvre, allez chercher le fichier *Datafish.csv*. La fenêtre suivante s'ouvre, dans laquelle vous allez pouvoir paramétrer certaines caractéristiques de votre fichier de données.



- Un nom pour votre jeu de données est généré automatiquement, par défaut, R reprend le nom sous lequel le fichier a été enregistré ou remplace par *dataset* ; attention si ce nom a déjà été utilisé ! Cela écrasera votre jeu de données *dataset* précédent. Il est possible de changer le nom dans *Name*.
- *Heading* indique si la première ligne de votre fichier correspond aux titres de vos variables.
- *Separator* permet d'indiquer sous quel format vos variables sont séparées dans les différentes colonnes de votre jeu de données ; cela varie selon le format du fichier : en .txt, le séparateur sera Tabulation, en .csv, le séparateur sera Semicolon (point-virgule), etc... Selon le logiciel, anglo-saxon ou non, le séparateur est différent.
- *Decimal* permet de préciser sous quelle forme est le marqueur décimal dans votre jeu de données (point ou virgule). Attention à bien préciser votre marqueur décimal, car R ne reconnaîtra pas directement la virgule comme marqueur décimal (par exemple, on écrira 3.2 dans un pays anglo-saxon, qui signifiera 3,2 en France). Ici, préciser *Comma* (virgule) car le marqueur décimal utilisé dans le fichier Datafish est la virgule.

### \* En choisissant From Text (readr)

La fenêtre suivante s'ouvre directement :



- Cliquer sur *Browse* en haut à droite, et choisir le fichier Datafish.csv.
- *Name* permet d'indiquer le nom que vous voulez donner à votre jeu de données.
- *Delimiter* permet d'indiquer le format du séparateur (Tab pour tabulation, etc...).
- Dans *Locale*, decimal mark permet de préciser la forme du marqueur décimal.
- *Code Preview* permet de visualiser quelles commandes vont finalement être exécutées.

- Cliquer enfin sur *Import* en bas à droite.

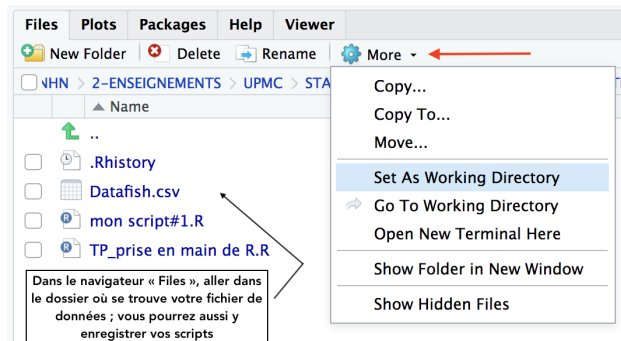
Une fois que vous avez importé les données Datafish, on les retrouve dans l'onglet *Environment*. Dans cet onglet, si vous cliquez sur la flèche à côté de Datafish, vous obtenez la description du jeu de données (ses différentes variables et leur nature). A cet endroit, en double-cliquant sur Datafish, vous affichez son contenu dans un onglet navigable (ce qui est plus pratique que de lire dans la console de R).



### Autre façon de procéder: à partir d'une commande exécutée dans le script

Pour cela, d'abord se placer au bon endroit dans l'onglet *Files* en bas à droite : il s'agit d'aller chercher le dossier où se trouve votre jeu de données.

Cliquer ensuite sur *More* dans cet onglet puis sur *Set as working directory* : cela permet de définir votre espace de travail, reconnu par R (notamment pour aller chercher des fichiers).



Ensuite, exécuter la commande :

```
Datafish <- read.csv("Datafish.csv", sep=";", dec=",")
#L'argument sep pour déterminer le séparateur et dec pour le marqueur décimal.
```

## Commencer à explorer le jeu de données

Ce jeu de données est un nouvel objet de R, appelé **Dataframe**. Il s'agit d'un tableau, où les individus statistiques figurent en lignes et les différentes variables mesurées ou observées sur ces individus figurent en colonnes.

Le fichier contient les tailles de deux groupes de poissons (adu1 et adu2) soumis à deux régimes alimentaires différents.

### Manipuler le jeu de données

- Utiliser les crochets `[ ]` et le signe d'extraction `$`

Une fois le fichier lu, on peut y accéder par ligne et colonne : on utilise les crochets, où on précise *[n° de ligne, n° de colonne]*.

```
Datafish[3,2] # 3ème ligne et 2ème colonne

# Ppouir corriger la valeur de l'individu 3, 1ère colonne, et lui assigner la valeur 55 :
Datafish[3,1]=55
```

```
Datafish[ , 2] # tous les individus, seulement la seconde colonne
```

```
Datafish[3 , ] # seulement la troisieme ligne, toutes les colonnes
```

```
Datafish$Groupe # le $ permet d'extraire le facteur "Groupe" dans l'objet "Datafish"
```

On peut également appliquer des fonction sur ces données :

```
names(Datafish) # donne le nom des colonnes
```

```
## [1] "Mesure" "Groupe"
```

```
mean(Datafish$Mes)
```

```
## [1] 49.80073
```

Notez l'usage astucieux des crochets pour extraire une modalité donnée d'une variable qualitative.

Cette variable s'appellera **facteur** (**factor**) dans R, et ses modalités **niveaux** (**levels**).

```
mean(Datafish$Mesure[Datafish$Groupe=="adu1"]) # moyenne de l'échantillon adu1
```

```
## [1] 54.45774
```

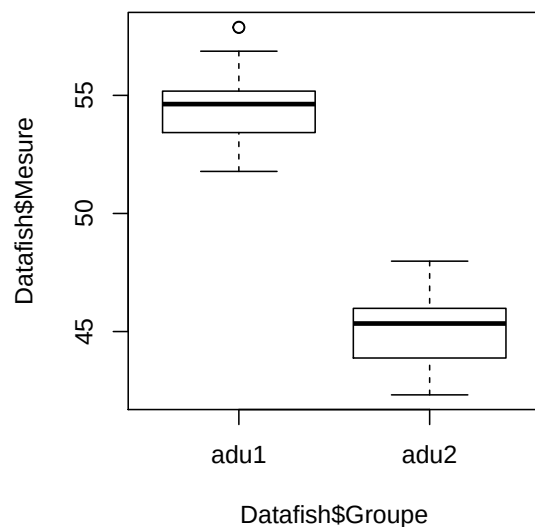
```
mean(Datafish$Mesure[Datafish$Groupe=="adu2"]) # moyenne de l'échantillon adu2
```

```
## [1] 45.14371
```

## Une première représentation graphique des données

La fonction `boxplot()` permet de réaliser des boîtes à moustaches. On peut utiliser cette fonction avec le symbole `~` (*tilde*), en plaçant la variable numérique avant le tilde et la variable qualitative (ou facteur) définissant nos groupes après : *variable numérique ~ variable qualitative*.

```
boxplot(Datafish$Mesure ~ Datafish$Groupe)
```



Autre façon d'écrire la même commande, avec l'argument `data =` qui permet d'éviter d'écrire `Datafish$`.

```
boxplot(Mesure ~ Groupe, data = Datafish)
```

De nombreux arguments graphiques existent. Voici comment changer les couleurs, avec l'argument `col =`.

```
boxplot(Mesure ~ Groupe, data = Datafish, col=c("green","blue"))
```

Voici comment changer le titre des axes :

```
boxplot(Datafish$Mesure ~ Datafish$Groupe, ylab="Taille des poissons (cm)", xlab="", col=c("green","blue"))
```

